

BAKERZ BITE

Developer Guide

Architecture • Code map • Data model • Storage • Cookbook • Chatbot • Testing • Known issues • Roadmap

1,892	337	16	34	4
lines in script.js	products	chatbot rules	test cases	open test items

Item	Detail
Audience	Developers who will change, extend, test or hand over the codebase
Source reviewed	cakes__2_.zip, top-level project copy (23 pages, css/style.css, js/products.js, js/script.js)
Stack	Static HTML5, CSS3, Bootstrap 5.3.3, jQuery 3.7.1, Vanilla JavaScript; no build step
Version	1.0 • 25 October 2026

Scope: this guide describes how the code works and how to change it safely. It is not a user manual and does not cover content decisions such as real prices, real hours or health and allergen wording.

Contents

1. About This Guide 3
2. Quick Start 3
3. Architecture 4
4. Code Map of js/script.js 5
5. Public API (window.BB) 6
6. Data Model 6
7. State and Storage 8
8. Rendering and HTML Safety 8
9. Events and Delegation 9
10. Forms and Validation Rules 9
11. Cookbook: Common Changes 10
12. Zoe's Matching Logic 11
13. CSS Architecture 12
14. Performance 12
15. Testing 13
16. Debugging Playbook 14
17. Known Issues and Technical Debt 14
18. Roadmap 15
19. Developer Release Checklist 15
Appendix A. File Index 16

Read this first

1. Work only in the top-level copy. The archive also contains an older nested copy in cakes/cakes/ with different versions of several files. Editing the wrong copy is the most likely way to lose work.

2. The test record is out of date. TEST-CASES.md says TC07 found 86 products between PKR 100 and 500, TC09 shows PKR 3,400 for product 1001, and TC10 expects a subtotal of PKR 19,400. Against the current data these are 142 products, PKR 2,000 and PKR 12,200 (6 items). Re-run those cases before relying on them (Section 15).

3. Zoe misroutes some questions. Keyword matching is substring-based, so "Do you have donuts?" matches the allergen rule through the word "nut". The fix is described in Section 12.

4. Two data files must agree. js/products.js is what the site reads. data/products.json is a reference copy that must be updated by hand. Category counts are typed in by hand inside products.js as well.

1. About This Guide

This guide is for developers who will maintain or extend the Bakerz Bite website. It explains the architecture, the code in js/script.js, the data model, the storage the site uses, the common tasks (with step lists), the chatbot's matching logic, the test record and the known issues. Line numbers refer to the top-level copy and will drift as the code changes.

File	Lines	Role
js/script.js	1,892	All behaviour: 15 sections plus the public API on window.BB
js/products.js	5,961	Data only: window.BB_CATEGORIES and window.BB_PRODUCTS
css/style.css	1,356	Theme tokens on :root, layout, responsive and print rules
***.html (23)**	about 600 each	Static markup; every page loads the same CSS and JS, and has its own body attributes
data/products.json	5,956	Reference copy of the catalogue (not read by the site)
TEST-CASES.md, README.md, PROJECT-REPORT.md	-	Test record, developer README and project report

Conventions used in this guide: **code** names appear in a monospace font; **section** means one of the banner-marked parts of js/script.js; a **line** is a product, offer, rule or other record in a data array.

2. Quick Start

Run it locally

The site has no build step. Serve the top-level folder with any static server so that relative paths and local storage behave as they do in production:

```
# option A: VS Code 'Open with Live Server' on index.html
# option B: from the project folder
python -m http.server 8000
# then open http://localhost:8000/
```

Before you change anything

Step	Why
Commit or copy the folder	There is no version control in the archive; keep a restore point.
Open DevTools (Console and Network)	Most failures show up as a first Console error or a 404 in Network.
Load index.html and one category page	Confirms that products render, the cart badge works and the chat opens.
Check localStorage in Application > Storage	Old bb_cart data can hide a bug. Clear it when testing a clean session.

Internet access is needed for Bootstrap, jQuery, Bootstrap Icons and Google Fonts. The site will load with degraded styling and no working Bootstrap components if the CDNs are blocked.

3. Architecture

There are three layers: the markup and stylesheet every page shares, the two script files, and browser storage. Nothing runs on a server.

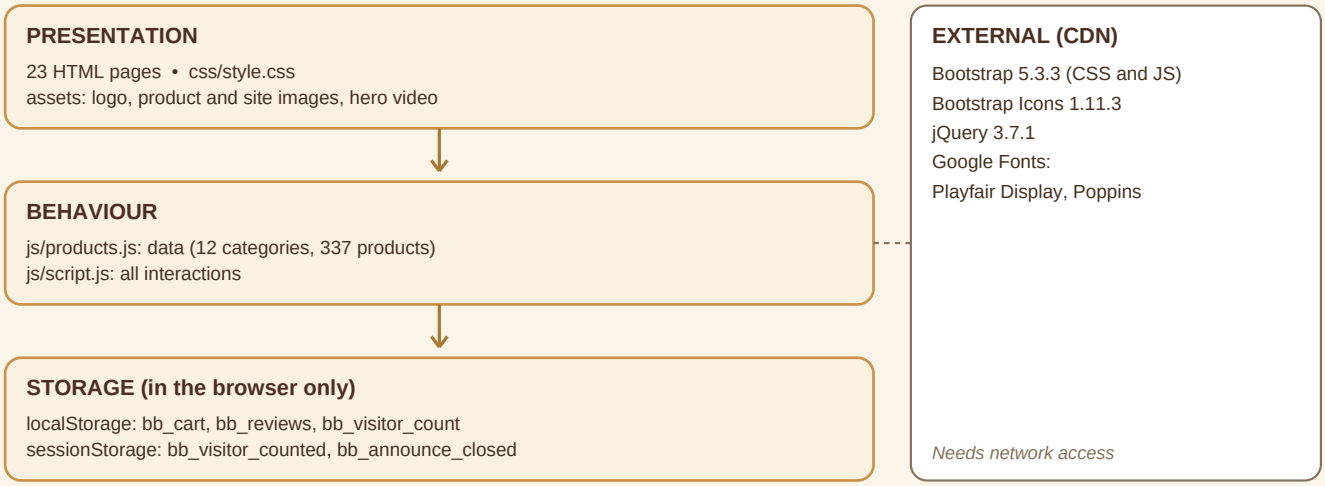


Figure 1. Layers and external dependencies.

Boot sequence on every page

Order	What happens	Where
1	Browser loads Bootstrap CSS, Bootstrap Icons, Google Fonts and css/style.css	<head> of each page
2	Bootstrap bundle and jQuery load, in that order	end of <body>, before the custom scripts
3	js/products.js runs and sets window.BB_CATEGORIES and window.BB_PRODUCTS	top of js/products.js
4	js/script.js runs inside an IIFE. It builds a byId lookup, reads bb_cart and drops invalid lines	script.js lines 1 to 80
5	initPage() calls about 20 init functions; each returns early when its elements are not on the page	script.js near line 1813
6	The page-specific renderers run, based on body attributes (data-page, data-category)	renderers listed in Section 4
7	bindEvents() attaches shared and delegated listeners	script.js near line 1666

Page detection

Each page declares what it is in the body tag. The script reads it through two helpers, page() and pageKey():

```
<body data-page="category" data-category="cakes" id="top">   <!-- cakes.html -->
<body data-page="menu" id="top">                              <!-- menu.html -->
<body data-page="customize" id="top">                           <!-- customize.html -->

// script.js
function page()    { return document.body.getAttribute("data-page") || ""; }
function pageKey() { return document.body.getAttribute("data-category") || ""; }
```

Page values in use: category (12 pages), menu, cart, customize, offers, gallery, about, feedback, faq, contact, sitemap and home. Every page loads the same script, so code that is page-specific must check for its own elements first.

4. Code Map of js/script.js

The file is organised into banner-marked sections. The table lists them in file order. 'Starts near' gives the banner line; the code itself begins one line below it. Use the banner titles to search.

Section	Starts near	What it owns	Main functions
Helpers	18	$, el, esc, fmt, starStr, page, pageKey	esc() is the escaping helper used everywhere
Storage	37	storage.get / set / remove wrappers around localStorage	try/catch on every call; missing storage returns the fallback
Toast	53	Pop-up notifications in #toastContainer	toast(message, type)
Cart	75	Cart array, totals, badge, add, update, remove, clear	addToCart, updateQty, removeFromCart, clearCart, cartTotals, persistCart
Catalogue	174	Menu and category grids: filters, sorting, Load More	filterCatalog, renderCatalog, populateCategoryFilter, pageSize
Product card	147	Card HTML with badges and rating stars	productCardHTML
Product modal	275	Detail pop-up for one product	openProductModal
Home	309	Category cards and featured products	renderCategoryCards, renderFeatured
Offers	341	Offer cards from the OFFERS array	offerCardHTML, renderOffers, offerToOrder
Order modal	395	Order form, quantity, total, sample receipt	openOrderModal, buildReceipt, initOrderModal
Gallery	560	Gallery grid, preview, lightbox	galleryItemHTML, renderGallery, openGalleryModal
FAQ	610	Accordion built from the FAQ array	renderFaq
Feedback and reviews	639	Review list, stored reviews, product options	storedReviews, renderReviews, populateFeedbackProducts
Custom cake builder	692	Options, price, summary, add to cart	customCakePrice, addCustomCake, syncCustomCake, collectAndValidate
Cart page	1045	Cart page rows and checkout summary	cartLineHTML, renderCartPage, openCheckout
Header, nav, ticker, misc	1122	Announcements, visitor counter, active links, timeline, bokeh, reveal, scroll UI, chatbot, ticker	initAnnouncements, initVisitorCounter, initNavActive, initChatbot, replyTo, initTicker
Form validation	1576	Shared field error display and page forms	setError, initFeedbackForm, initContactForm
Event bindings	1663	Shared and delegated listeners	bindEvents
Init	1761	Startup sequence	initPage
Public API	1865	The window.BB object	see Section 5

The bokeh background and the preloader live in the header and misc section and the animation section near line 1276.

5. Public API (window.BB)

Because script.js is wrapped in an IIFE, the only way to reach its internals from outside is the object assigned to window.BB. Use it from the browser console to inspect and test behaviour without editing the code.

Member	Type	Purpose
products, categories	array	The catalogue arrays from products.js (same objects, not copies)
storage	object	get(key, fallback), set(key, value), remove(key) over localStorage
cart	array	Live cart array; mutate only through the functions below
addToCart(id, qty)	function	Adds a product line or increases its quantity (max 99); rejects unavailable products
updateQty(key, qty)	function	Sets a line's quantity; 0 or less removes the line
removeFromCart(key), clearCart()	function	Remove one line or all lines; both persist and refresh the badge
cartTotals(), cartCount()	function	Returns subtotal, items and total; cartCount returns the item total
filterCatalog, catalogState	function, object	Current catalogue filter state and the function that applies it
openProductModal(id), fmt(n)	function	Opens the product pop-up; formats numbers as PKR
renderCartPage()	function	Re-renders the cart page if it is open
customCakeOptions, customCakePrice(spec)	object, function	Options and the price rule for a custom cake specification
addCustomCake(spec, opts), openCustomOrder(spec)	function	Adds a configured cake to the cart; opens the order flow
isCustomLine, lineKey, lineUnitPrice, lineName	function	Helpers that treat product and custom lines the same way

Console examples

```
BB.cartCount()                      // items in the cart
BB.cartTotals()                     // { subtotal, items, total }
BB.storage.get("bb_cart", [])        // raw stored cart
BB.customCakePrice({ size: "2", tiers: "2", flavor: "red-velvet",
                  frosting: "ganache", filling: "fresh-fruit", extras: [] })
BB.products.filter(p => p.cat === "cakes").length     // 50
BB.addToCart(1001, 2)               // adds two Chocolate Fudge Cakes
```

Note: BB.cart is the live array, so a direct edit is persisted only when a later function call runs persistCart(). Prefer the functions.

6. Data Model

Every record below lives in a JavaScript array. Field names are case-sensitive and must match exactly, because the renderers read them by name.

Product (js/products.js, BB_PRODUCTS)

Field	Type	Used for	Rule
id	number	Cart key, lookup, modal	Unique across all 337 products (verified)
name	string	Card title, cart line, receipt	Plain text; escaped on output
category, cat	string, string	category = display label; cat = key used in URLs and filters	cat must equal a key in BB_CATEGORIES
price	number	All totals (PKR)	Whole number; range in data is 150 to 3,200
description, ingredients, allergens	string	Product modal and card text	Allergen text is advisory only
available	boolean	Blocks add to cart when false	Unavailable items still show, with a badge

Field	Type	Used for	Rule
featured, bestseller	boolean	Badges and the Home page highlights	Optional
dietary	array	Vegetarian and similar flags	Empty array when none apply
rating, reviews	number	Star display and review count	Sample values, not verified
image	string path	Card and cart thumbnail	File must exist; falls back to placeholder.svg on error

Category (BB_CATEGORIES)

Fields: key (URL and filter key, for example cakes), label, icon (a Bootstrap Icons class), blurb, count, image and href. count is typed in by hand and must equal the number of products whose cat matches key. It is correct today for all 12 categories, but nothing enforces it.

Offer (OFFERS, script.js line 344)

Fields: id (unique), price, badge (for example '-20%' or 'Custom'), img, title, desc, terms (array of strings) and dates (sample text). The first three offers appear on the Home page.

Gallery item, FAQ item and review

Record	Fields	Notes
Gallery (GALLERY, line 563)	img, cap, tags	20 items; tags is a space-separated string used by the filter buttons
FAQ (FAQ, line 613)	q, a	10 items; a is plain text with \n line breaks
Seed review (SEED_REVIEWS, line 642)	name, product, rating, text, date, demo	4 items with demo: true; these are never written to storage
Stored review (bb_reviews)	name, email, product, rating, text, date, demo	demo is false; date is the text 'Just now' when saved

Custom cake options (CUSTOM_CAKE_OPTIONS, line 695)

occasions is a plain string array. flavors, tiers, frostings, fillings and extras are arrays of { id, label, add } and sizes are arrays of { id, label, base, serves }. Ids must be unique inside each array, because findOption() returns the first match.

Cart lines (bb_cart)

Line type	Shape	Key
Product	{ id, qty }	String form of id
Custom cake	{ kind: "custom", key, qty, price, name, image, details: [{ k, v }] }	The generated key (prefix c-), unique per addition

7. State and Storage

Key	Storage	Written by	Read by
bb_cart	localStorage	persistCart() after every cart change	Cart load at startup; cart pages; badge
bb_reviews	localStorage	Feedback form submit	storedReviews() on the Feedback page
bb_visitor_count	localStorage	bumpVisitorCount()	initVisitorCounter(); header counter
bb_visitor_counted	sessionStorage	bumpVisitorCount() once per session	same
bb_announce_closed	sessionStorage	Announcement close button	initAnnouncements()

Cart lifecycle

At startup the cart is read from bb_cart. The loader keeps a line only if it is a custom line, or if its id exists in the product list. Every change calls persistCart(), which writes the whole array. There is no version field, so a future change to the line shape needs a migration step in the loader.

Trust boundary

> **Known weakness:** the loader does not check custom lines. Their price, name and details come from storage as they were written, so anyone can edit bb_cart in DevTools and change the price of a custom cake. This is acceptable for a demonstration. A real shop must price every line from the server.

Clearing state while testing

```
localStorage.removeItem('bb_cart');
localStorage.removeItem('bb_reviews');
localStorage.removeItem('bb_visitor_count');
sessionStorage.clear();
location.reload();
```

8. Rendering and HTML Safety

Most of the script builds HTML strings and assigns them with innerHTML. Any value that comes from a visitor, from storage or from data that could change must go through esc() first. esc() replaces the characters &, <, >, double quote and single quote.

> **Rule for new code:** escape every interpolated value that is not a constant you typed yourself. Use textContent instead of innerHTML wherever the value is plain text.

Where user or stored text appears	Escaped?
Review name, product, text (carousel)	Yes, esc() in renderReviews
Toast message	Yes, esc(message)
Cart line name, image, details (k and v)	Yes, in cartLineHTML
Custom cake message in the summary	Yes, via details rows
Product name and description in cards	Yes; data is trusted but still escaped
Announcement text	Yes, esc(ANNOUNCEMENTS[i])

Images that fail to load are swapped for assets/images/placeholder.svg by initImageFallback(). Any new <img> should carry a data-fallback attribute pointing to that file, and a meaningful alt.

Inline styles in JavaScript

renderFeatured() adds some badges with inline style strings. This works, but styling belongs in css/style.css. Moving those badges to classes is a low-risk clean-up (see Section 17).

9. Events and Delegation

bindEvents() attaches one listener per shared control and uses event delegation for controls that are rendered later (product cards, cart rows). Delegated controls are identified by data attributes and classes rather than ids, so re-rendering does not break them.

Control	Identifier	Action
Add to cart on a card	data-id on the add button	addToCart(id, 1)
Quantity plus and minus on the cart page	class cart-qty-plus / cart-qty-minus with data-line	updateQty(key, qty ± 1)
Quantity typed on the cart page	class cart-qty-input with data-line	updateQty(key, value)
Remove line	class cart-remove-btn with data-line	removeFromCart(key)
Order quantity in the modal	ids odQtyMinus, odQtyPlus, odQty	orderQty() and updateOrderTotal()
Custom cake form	ids ccOccasion, ccFlavor, ccSize, ccTiers, ccFrosting, ccFilling, ccExtras	syncCustomCake() on every change

Keep new listeners inside bindEvents() so that the order of setup stays in one place. Avoid binding the same element twice; a second bind causes duplicate toasts and double counting.

10. Forms and Validation Rules

Validation runs in the browser only. The rules below are the ones in the current source. Error text is written into a box beside the field by setError(id, message), which also toggles a CSS class on the field.

Form	Field	Rule (current code)
Feedback	email	Optional; if filled, must match /^[^\s@]+@[^\s@]+\.[^\s@]+$/
Feedback	review text	At least 5 characters after trimming
Contact	email	Required; same pattern as above
Contact	phone	Optional; if filled, /^[0-9+\-\s()]{7,}$/
Contact	message	At least 10 characters after trimming
Order (product and cart)	phone	Required; same phone pattern
Order (product and cart)	email	Required; same email pattern
Order	quantity	Clamped to 1 to 99 in orderQty()
Custom cake	message	At most 120 characters; error if longer
Custom cake	size	Required before a cake can be added (addCustomCake returns false otherwise)
Any add to cart	qty	Parsed as an integer; less than 1 becomes 1; capped at 99

> **Inconsistency to be aware of:** the feedback review needs 5 characters but the contact message needs 10. This is probably deliberate but is not documented anywhere. Decide on one rule per form and record it.

11. Cookbook: Common Changes

Each recipe lists the files to touch in order. Check every item on the list, then run the tests in Section 15 for the area you changed.

11.1 Add a product

Step	Action
1	Add an object to BB_PRODUCTS in js/products.js. Use the next free id (ids must be unique).
2	Set cat to an existing category key, and category to its label exactly as in BB_CATEGORIES.
3	Put the photo at assets/images/products/<category>/<name>.jpg and make sure the path matches image exactly.
4	Add 1 to the count of that category in BB_CATEGORIES.
5	Add the same product to data/products.json, or accept that the reference copy is stale.
6	Open the category page and the Full Menu. Check the card, modal, add to cart and the Load More count.

11.2 Add a category

Step	Action
1	Add an entry to BB_CATEGORIES with key, label, icon, blurb, count, image and href.
2	Copy cakes.html to <key>.html. Change the title, the body data-category to the new key and the page heading.
3	Add the new link to the Categories dropdown in all 23 pages. The dropdown is static HTML, not generated by script.js.
4	Add the link to the footer's Categories list in every page and to sitemap.html (Shop by Category, currently 13 pages).
5	Update the FAQ answer about the number of categories, and the Zoe categories rule (Section 12).
6	Confirm the filter dropdown on the Full Menu includes the new label (populateCategoryFilter reads BB_CATEGORIES).

11.3 Add or change an offer

Add an object to OFFERS in js/script.js (around line 344). Use a new unique id. Keep terms as short strings. Set img to a real file. The first three entries appear on the Home page, so order the array deliberately. Test the Offers page and the order popup for the new offer.

11.4 Add a custom cake option

Step	Action
1	Add { id, label, add } to the right array in CUSTOM_CAKE_OPTIONS (script.js line 695). Keep ids unique in that array.
2	Check that the matching group container exists in customize.html: ccOccasionGroup, ccFlavorGroup, ccSizeGroup, ccTiersGroup, ccFrostingGroup, ccFillingGroup or ccExtrasGroup.
3	Use the browser console: BB.customCakePrice(spec) with the new id should add exactly the value you entered.
4	Check the price table on the page. It is rendered from the same arrays, so it should update without HTML changes.

11.5 Add or change a chatbot rule

Add { k: [...], a: '...' } to RULES (script.js line 1428). Keywords are lower-case substrings. Read Section 12 first: a new keyword can match inside a word that you did not mean, and rule order decides ties.

11.6 Add an FAQ entry

Add { q, a } to FAQ (line 613). Keep answers short and consistent with the other pages. The accordion is rebuilt on load, so no HTML change is needed. Update the FAQ count in any document that states it.

11.7 Add a new page

Step	Action
1	Copy an existing simple page (for example faq.html). Set a unique body data-page value.
2	Add the page to the navbar, the footer and sitemap.html in all 23 existing pages plus the new one.

Step	Action
3	If the page needs JavaScript, add a guarded init function and call it from initPage(). Return early when its element id is missing.
4	Add the page to TEST-CASES.md with its link, responsive and console checks.

11.8 Change the theme

Colours are CSS variables on :root in css/style.css (see Section 13). Change them there, not in individual rules. The bokeh colours are set in the animation section of script.js near line 1276, so a palette change should also update that list. Re-check contrast for small muted text after any change.

12. Zoe's Matching Logic

Zoe reads the message in lower case and counts how many keywords from each rule appear anywhere in it. The rule with the highest count wins. A tie keeps the rule that appears first in the RULES array, because the comparison is strict. If no keyword matches, the FALLBACK reply is used. The rules are checked in order and there are 16 of them.

```
function replyTo(raw) {
  var msg = String(raw || "").toLowerCase();
  var best = null, bestScore = 0;
  for (var i = 0; i < RULES.length; i++) {
    var score = 0;
    for (var j = 0; j < RULES[i].k.length; j++) {
      if (msg.indexOf(RULES[i].k[j]) !== -1) score++;   // plain substring test
    }
    if (score > bestScore) { bestScore = score; best = RULES[i]; }
  }
  return bestScore > 0 ? best.a : FALLBACK;
}
```

Problem: substring matching

The test is msg.indexOf(keyword), so a keyword matches inside a longer word. I ran the current RULES against sample questions to confirm the results below.

Question	Rule chosen	What is wrong
Do you have donuts?	Allergen rule (score 2)	'nut' and 'nuts' match inside 'donuts'. The categories rule (which has 'donut') scores only 1, so the allergen reply wins.
What donuts do you sell?	Allergen rule (score 2)	Same cause. 'sell' should lead to the menu rule, but the allergen score is higher.
Is your bread vegan?	Allergen rule (score 1)	'vegan' is correct, but the tie with the categories rule goes to the earlier rule. Arguable.
Do you sometimes have offers?	Offers rule (score 1)	Right answer by accident: 'time' in 'sometimes' also scores the hours rule, but offers comes first.
What time do you open?	Hours rule (score 2)	Correct.

Suggested fix

Match whole words with a regular expression that requires a word boundary before and after each keyword. Escape the keyword so that characters like the hyphen and the dot are literal. Keep the highest-score rule, and make the tie-break explicit (for example by longest matched keyword).

```
function keywordHit(msg, k) {
  var safe = k.replace(/[.*+?^${}()|[\]\\]/g, "\\$&");
  return new RegExp("\\b" + safe + "\\b").test(msg);
}
// then in replyTo(): if (keywordHit(msg, RULES[i].k[j])) score++;
```

> After any change, re-run the eight sample questions above and add the donut question to the test list. Note that a whole-word match will not match 'sometimes' for 'time', which is the intended behaviour.

13. CSS Architecture

css/style.css is one file of about 1,356 lines. It starts with the design tokens on :root (the token list is in Section 7 of the Full Project Documentation), then the shared components, then page sections, then media queries and print rules.

Concern	Where it lives	Notes
Colour and type tokens	:root variables (--bg, --chocolate, --gold, --rose, --deep, --text, --text-muted and others)	Change here first; nothing else should hard-code these colours
Component names	Classes prefixed bb- (bb-navbar, bb-btn, bb-card, bb-chat, bb-toast)	Keep the prefix for new shared components
Responsive breakpoints	@media max-width 575.98, 767.98, 991.98, 1199.98 px; min-width 1200 and 1400 px ranges	Use the same breakpoints; do not add new ones without a reason
Reduced motion	@media (prefers-reduced-motion: reduce) sets animations to near-zero duration	Add new animations to this rule's coverage
Print	@media print	Keep print rules minimal; the cart and receipts are the only printable views

Fonts are loaded from Google Fonts with a fallback stack defined through --font-head and --font-body. If the font request fails, the fallback fonts are used; check that the layout still fits.

14. Performance

Area	Current state	Suggested action
products.js	About 5,961 lines (roughly 200 KB) loaded on every page, including the home page and FAQ	Load only on pages that show products, or split the data into per-category files
Product images	227 unique product image files plus site and team photos	Compress and size to the displayed dimensions; keep loading="lazy" on card images
Hero video	hero.mp4 is about 1.7 MB and plays on every page	Consider a smaller file, and make sure the poster image shows when autoplay is blocked
Bokeh animation	Runs on every page with many layered elements	Measure on a low-end phone; reduce elements or respect reduced motion more strongly
Third-party CDNs	Bootstrap, jQuery, icons and fonts from CDNs	Consider local copies for a fixed demo; add integrity hashes if versions are pinned

Measure before and after. Use the Performance panel in DevTools and record the first contentful paint on a throttled phone profile.

15. Testing

The test record in TEST-CASES.md was produced with Node.js and jsdom against a local static server, plus static validators. It is not an automated test suite you can run from the project, and some of its results no longer match the data.

Status of the four records that depend on data

Case	Record says	Current code gives	Action
TC07 price filter 100 to 500	86 matching products	142 matching products	Re-run; update the record
TC09 modal for id 1001	PKR 3,400	PKR 2,000	Re-run; update the record
TC10 cart subtotal	PKR 19,400 with 6 items	PKR 12,200 with 6 items (5 x 2,000 + 1 x 2,200)	Re-run; the record's subtotal does not follow from its own steps
TC11 cart persistence	bb_cart holds the line	Consistent with the code	Re-run anyway after data changes

Why this matters: a passing record that describes older data gives false confidence. Treat the record as a checklist, not as proof that the current build passes.

Open items

TC02 (responsive navbar), TC21 (geolocation, partial), TC22 (browser compatibility) and TC23 (responsive layout) need a real browser or a device. The results so far come from code inspection and from jsdom, which does not render layout.

Suggested test layers

Layer	What to test	How
Pure logic	customCakePrice, cartTotals, lineKey, keywordHit (after the fix)	Extract to a module, or test through window.BB in a headless browser
Data integrity	Unique ids, category keys exist, counts match, image files exist	A short Node script that reads products.js and checks each rule
Browser flows	Add to cart, change quantity, refresh, order, feedback, contact, chat	Playwright or a manual script run on Chrome and a mobile profile
Visual and responsive	Breakpoints at 360, 768, 1024, 1440 px	Screenshot comparison or a manual checklist
Accessibility	Keyboard use, focus, contrast, skip link	axe or Lighthouse plus manual keyboard testing

> **Suggested first script:** a data-integrity check that fails the build if a product id repeats, a cat value has no matching category, a count is wrong or an image path is missing. Those four checks would have caught the stale counts in this record.

16. Debugging Playbook

Symptom	Likely cause	Where to look
Product grid is empty	products.js did not load, or it loaded after script.js	Network tab for products.js; script order at the end of <body>
Cart badge stays at 0	Badge id missing on the page, or persistCart failing	Element cartCountBadge; Console for storage errors
Cart empty after refresh	bb_cart rejected by the loader filter (unknown id)	BB.storage.get('bb_cart', []) before and after reload
Category page shows all products	body data-category missing or misspelt	The body tag and the cat values in products.js
Chat answers the wrong topic	Substring match and rule order	Section 12; run replyTo() with the question in the console
Feedback review does not appear	Validation failed silently, or stored value is not an array	Form error boxes; bb_reviews in Application > Storage
Image shows the placeholder	Path wrong, file missing or different case	Network 404; exact path in products.js
Custom cake total is NaN or 0	Size not chosen, or an option id missing from CUSTOM_CAKE_OPTIONS	BB.customCakePrice(spec) with the same spec
Announcement stops changing	Interval cleared or element missing	initAnnouncements; the announcementText id

Useful one-liners: BB.products.length should be 337. BB.categories.length should be 12. Both values should match the reference in Section 15.

17. Known Issues and Technical Debt

Each item is confirmed in the current code. Severity reflects impact on the demonstration if left unfixed.

ID	Issue	Where	Severity	Suggested fix
K1	Zoe matches keywords inside longer words ('donuts' triggers the allergen rule)	script.js, replyTo()	Medium	Whole-word matching (Section 12)
K2	TC07, TC09 and TC10 records do not match current data	TEST-CASES.md	High	Re-run and update; add data-integrity checks
K3	Second, older copy of the site inside the archive	cakes/cakes/	High	Delete or archive; keep one canonical copy
K4	Custom cart lines are not re-priced when loaded	script.js line 80, cart loader	Medium	Re-price from CUSTOM_CAKE_OPTIONS on load, or move pricing to a server
K5	Category counts are typed by hand and products.json is a manual copy	products.js, products.json	Medium	Generate both from one source; add a check
K6	Navbar, footer and category dropdown are repeated in 23 HTML files	*.html	Medium	Include a shared fragment, or generate pages from a template
K7	Zoe's hours and currency differ from the contact page and the FAQ	RULES; footer; contact.html	Medium	Use one source of truth for hours and currency
K8	Feedback needs 5 characters, contact message needs 10, with no rule written down	script.js, forms	Low	Pick one rule per form and document it
K9	Announcement interval is never cleared	initAnnouncements()	Low	Keep a handle and clear it on page hide, or leave as is
K10	Some badges use inline style strings	renderFeatured()	Low	Move to CSS classes
K11	Internal functions are only reachable through window.BB	IIFE structure	Low	Split into ES modules when the test suite is added

ID	Issue	Where	Severity	Suggested fix
K12	Docs say 22 pages; there are 23	README, PROJECT-REPORT, TEST-CASES	Low	Update the documents

Counting the table: 12 items, 2 High, 5 Medium, 5 Low. The two High items are data and repository problems, not code bugs, and should be fixed before any other release decision.

18. Roadmap

Short term (before the next release)

Task	Closes
Re-run TC07, TC09, TC10 and TC11 on the current data and update TEST-CASES.md	K2
Delete or archive the nested copy and note the decision in the README	K3
Fix Zoe's matching with whole-word matching and align hours and currency	K1, K7
Add the data-integrity script from Section 15 and run it before each release	K5

Medium term

Generate products.json and the category counts from products.js, or the reverse. Move the navbar, footer and dropdown into one shared fragment, or generate the 23 pages from a small template. Re-price custom cart lines from the option table on load (K4). Move inline badge styles into CSS (K10).

Long term (if the site is to take real orders)

Put pricing, stock, orders, contact messages and reviews behind a server, with server-side validation and a privacy notice. Keep the browser code as a presentation layer only. This is a rebuild of the data flow, not a patch, so plan it as a separate project.

19. Developer Release Checklist

Check	Done
Changed only the canonical copy; nested copy untouched or archived	[ ]
Data integrity: unique ids, category keys, counts, image files	[ ]
No new console errors on home, a category page, menu, cart, customize, contact	[ ]
Cart add, change quantity, remove, clear and refresh all work	[ ]
Custom cake total matches the option table	[ ]
Forms show errors and do not claim a message was sent	[ ]
Zoe answers the eight sample questions in Section 12 correctly	[ ]
Breakpoints checked at 360, 768, 1024, 1440 px	[ ]
Test record updated, with the date and the data version tested	[ ]
Version, date and reviewer recorded in the change log	[ ]

Appendix A. File Index

Path	Purpose
index.html and the other 22 pages	Markup, body attributes, navbar, footer, page-specific containers
css/style.css	Tokens, components, responsive and print rules
js/products.js	Catalogue data (BB_CATEGORIES, BB_PRODUCTS)
js/script.js	All behaviour and the window.BB API
data/products.json	Reference copy of the catalogue
assets/logo/	Badge (PNG), logo and favicon (SVG)
assets/images/	Product, site and team photographs; placeholder.svg; manifest and CREDITS files
assets/video/	hero.mp4 and hero-poster.jpg
README.md, PROJECT-REPORT.md, TEST-CASES.md	Documentation and test record

> **Thank you** for maintaining Bakerz Bite. Keep the data honest, escape everything, and test before you release.